Dataset & Preprocessing

WikiText-103 · GPT-2 Tokenizer · Causal LM Setup

Transformer LMs without Positional Encodings Still Learn Positional Information

Patra · Das · Ahmed · Mondal · Kumar

DATASET

WikiText-103

Large-scale Wikipedia language modeling benchmark · Hugging Face datasets library

314M
Total Tokens
raw text, no normalization

1.8M
Train Examples
Wikipedia article sections

3.7K
Validation
examples

4.3K
Test
examples

- Raw variant: original Wikipedia formatting — no symbol substitution
- Loaded via HuggingFace datasets library using load_dataset('wikitext', 'wikitext-103-raw-v1')
- Three splits (train / validation / test) loaded independently
- Text field per row — each row is one Wikipedia article section

CONFIGURATION

Config System

YAML files → Python dataclasses · config.py + configs/.yaml*

DatasetConfig

name: wikitext

config: wikitext-103-raw-v1

tokenizer: gpt2

seq_len: 512

stride: 256 (train)

max_tokens: null

ModelConfig

d_model: 512

n_heads: 8

n_layers: 8

d_ff: 2048

dropout: 0.1

pos_encoding: {none|sine|learned|alibi}

TrainConfig

batch_size: 16 (8 on Kaggle)

grad_accum: 1 (4 on Kaggle)

lr: 3e-4

optimizer: adamw

scheduler: cosine

warmup_steps: 500

load_config('configs/wikitext103.yaml') → merges YAML into Config dataclass via _merge_dataclass()

PREPROCESSING — STEP 1

Loading & Tokenization

1. **Load GPT-2 Tokenizer**
   50,257 token BPE vocabulary. EOS & PAD tokens ensured.

2. **Load Dataset Splits**
   train / validation / test via HuggingFace load_dataset()

3. **Filter Empty Texts**
   Empty or whitespace-only rows skipped before tokenizing

4. **Batch Tokenization**
   1,000 articles per batch · no_special_tokens · no padding

5. **Append EOS Separator**
   EOS token appended after every document — marks boundary

6. **Flatten to 1D Tensor**
   All token IDs concatenated into one contiguous int64 tensor

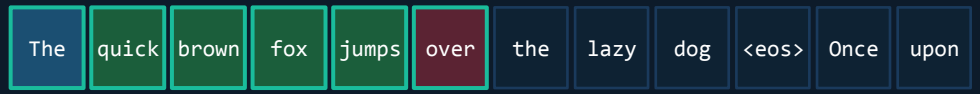
_encode_text() — core logic

```
def tokenize_fn(examples):
  texts = [t for t in examples['text']
           if t and t.strip()]
  encoded = tokenizer(texts,
    add_special_tokens=False)
  for ids in encoded['input_ids']:
    ids_batched.extend(ids)
    ids_batched.append(eos_id)  # <-
return torch.from_numpy(arr)
```

PREPROCESSING — STEP 2

CausalLMDataset · Sliding Window

Converts flat token stream into (x, y) pairs for next-token prediction



x (input, seq_len=5 shown)

y (target, shifted by 1)

stride = 256 (50% overlap for train) → more examples, richer context exposure

seq_len	512 tokens per example	**x**	tokens[i : i+512]
stride (train)	256 → 50% overlap	**y**	tokens[i+1 : i+513]
stride (val/test)	512 → no overlap	**# samples**	⌊(N − 513) / stride⌋ + 1

No padding needed — flat stream guarantees every window is exactly seq_len tokens

DataLoaders

Wraps CausalLMDataset for efficient batched GPU training

train_dl

shuffle	True
drop_last	True
batch_size	16 (8 on Kaggle)
grad_accum	1 (4 on Kaggle)

Shuffled overlapping windows prevent order memorization

val_dl

shuffle	False
drop_last	False
stride	512 (no overlap)
purpose	Monitor val loss / ppl

No overlap avoids double-counting tokens in perplexity

test_dl

shuffle	False
drop_last	False
stride	512 (no overlap)
purpose	Final evaluation only

Loaded but not used until training is complete

All loaders: num_workers = 2–4 · pin_memory = True · non_blocking GPU transfer

SUMMARY

End-to-End Pipeline

Raw Text
WikiText-103 splits

Tokenize
GPT-2 BPE · EOS separator

Flat Stream
1D int64 tensor

Slide Windows
seq=512 · stride=256/512

(x, y) Pairs
Causal next-token targets

DataLoader
Batched · shuffled · GPU-ready

No Padding

Flat stream + fixed windows — every batch is fully dense

Tied Weights

Token embedding matrix shared with LM head — reduces parameters

50% Overlap (train)

Each token seen in multiple contexts → richer supervision signal

AMP Training

Automatic Mixed Precision — halves VRAM, speeds up training

EOS as Doc Separator

Model learns clean document boundaries without extra tokens

Kaggle-Ready Config

grad_accum=4, batch=8 simulates effective batch 32 on T4/P100